DevOps CI/CD Project Report

Task Manager API - Production-Grade Pipeline Implementation

---

1. Problem Background & Motivation

1.1 The Challenge of Modern Software Delivery

In today's fast-paced software development environment, organizations face significant challenges:

- **Manual deployments** are error-prone and time-consuming
- **Security vulnerabilities** often go undetected until production
- **Code quality issues** accumulate as technical debt
- **Inconsistent environments** lead to "works on my machine" problems

1.2 Why CI/CD?

Continuous Integration and Continuous Deployment (CI/CD) addresses these challenges by:

1. **Automating repetitive tasks** - Builds, tests, and deployments run automatically
2. **Shifting security left** - Finding vulnerabilities early when they're cheaper to fix
3. **Ensuring consistency** - Same process runs every time, reducing human error
4. **Enabling rapid feedback** - Developers know immediately if their changes break something

1.3 Project Objective

This project implements a **production-grade CI/CD pipeline** for a Java REST API that demonstrates:

- Automated build and test processes
- Security scanning at multiple levels (SAST, SCA, container scanning)
- Container-based deployment
- Kubernetes orchestration

---

2. Application Overview

2.1 Task Manager API

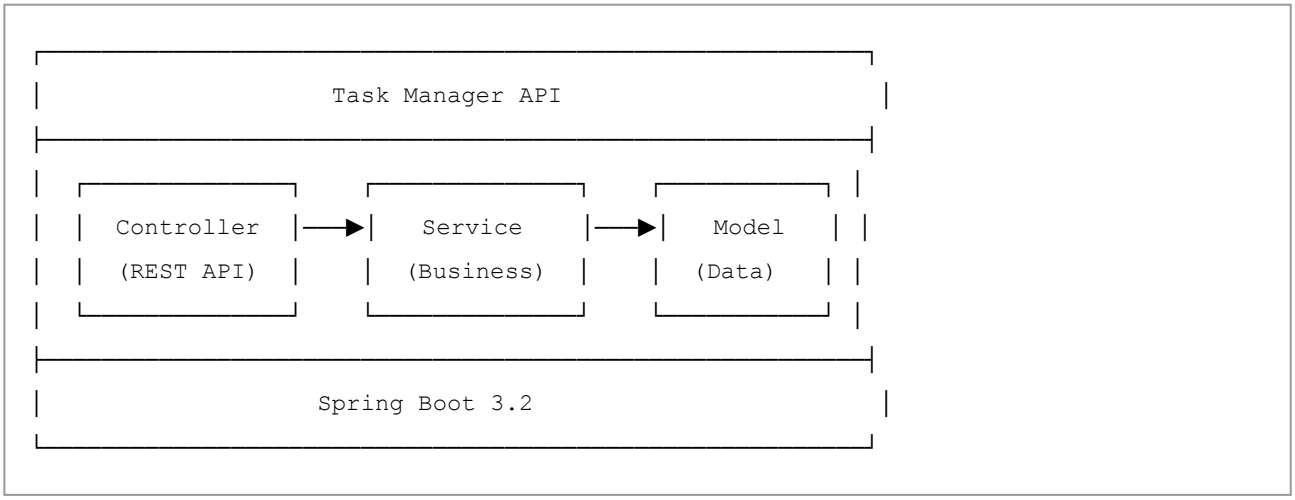
A simple REST API built with Spring Boot for managing tasks. The application provides CRUD operations for task management.

Technology Stack: | Component | Technology | Version | |-----------|------------|---------| | Language | Java | 17 | | Framework | Spring Boot | 3.2.0 | | Build Tool | Maven | 3.9+ | | Container | Docker | - | | Orchestration | Kubernetes | - |

2.2 API Endpoints

Method	Endpoint	Description
GET	/api/tasks	List all tasks
GET	/api/tasks/	Get specific task
POST	/api/tasks	Create new task
PUT	/api/tasks/	Update task
DELETE	/api/tasks/	Delete task
GET	/api/tasks/health	Health check

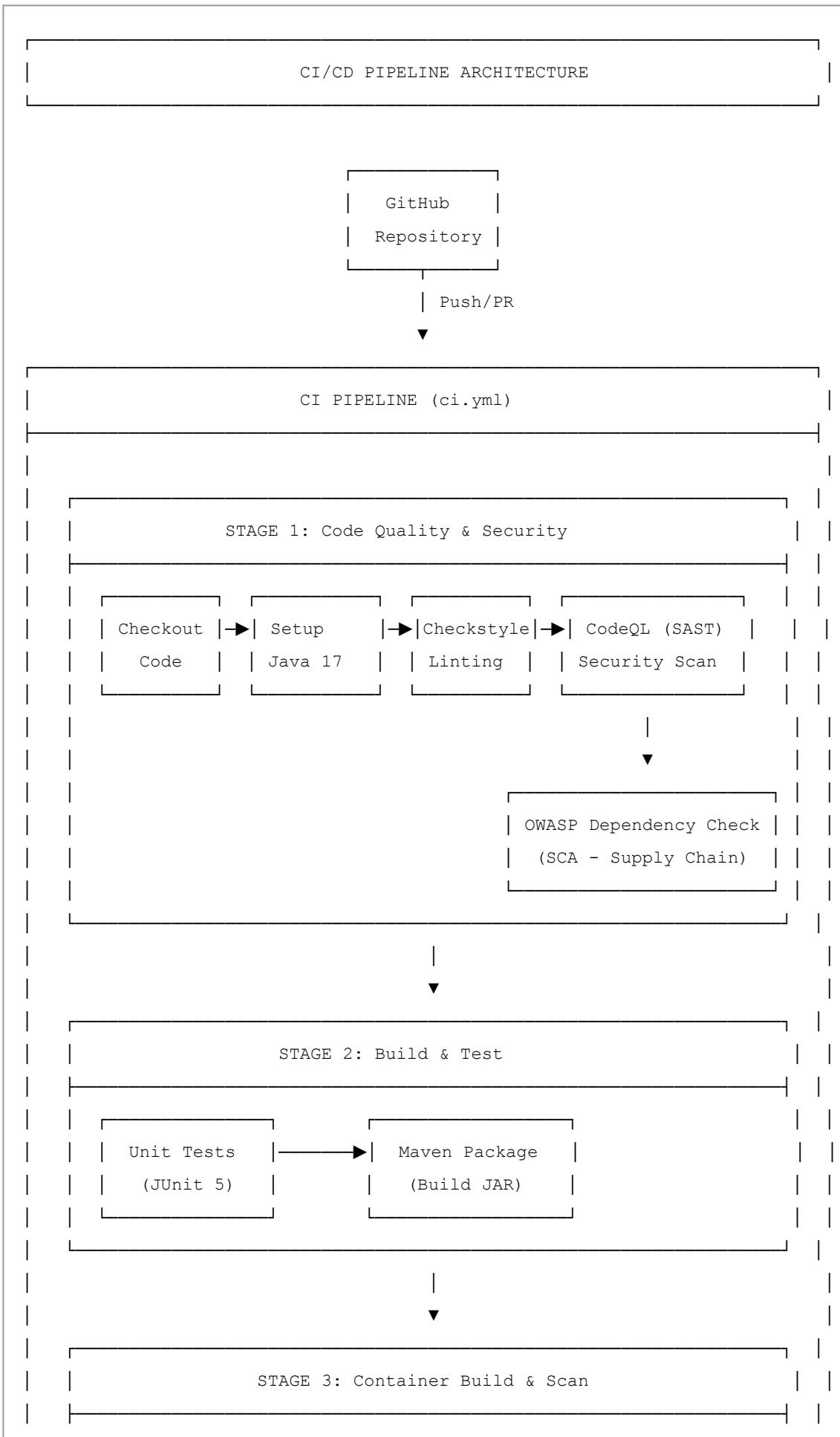
2.3 Application Architecture

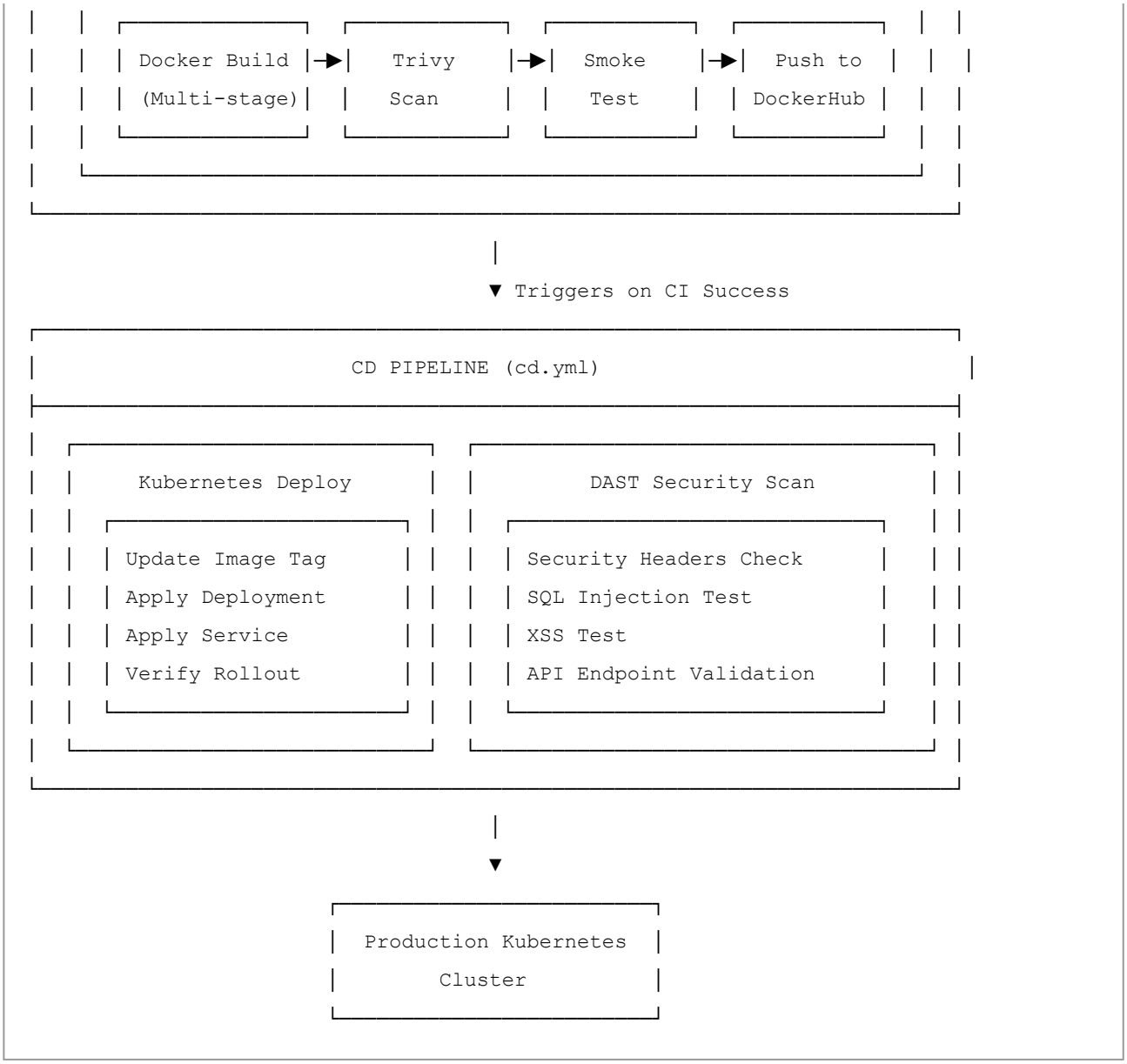


---

3. CI/CD Architecture Diagram

3.1 Complete Pipeline Flow


CI/CD PIPELINE ARCHITECTURE
GitHub Repository
Push/PR
CI PIPELINE (ci.yml)
STAGE 1: Code Quality & Security
Checkout Code
Setup Java 17
Checkstyle
Linting
CodeQL (SAST)
Security Scan
OWASP Dependency Check
(SCA - Supply Chain)
STAGE 2: Build & Test
Unit Tests
(JUnit 5)
Maven Package
(Build JAR)
STAGE 3: Container Build & Scan



4. CI/CD Pipeline Design & Stages

4.1 CI Pipeline Stages

#	Stage	Tool	Purpose	Risk Mitigated
1	Checkout	actions/checkout	Retrieve source code	Missing code
2	Setup Java	actions/setup-java	Install JDK 17	Build environment issues
3	Linting	Checkstyle	Enforce coding standards	Technical debt
4	SAST	CodeQL	Static security analysis	OWASP Top 10 vulnerabilities
5	SCA	OWASP Dependency Check	Dependency vulnerabilities	Supply chain attacks
6	Unit Tests	JUnit 5	Validate business logic	Regressions

#	Stage	Tool	Purpose	Risk Mitigated
7	Build	Maven	Package application	Build failures
8	Docker Build	Docker	Create container image	Deployment inconsistencies
9	Container Scan	Trivy	Image vulnerabilities	Vulnerable containers
10	Smoke Test	Docker	Runtime validation	Non-runnable images
11	Push	DockerHub	Publish trusted image	Image availability

4.2 Why Each Stage Matters

Linting (Checkstyle)

- Prevents accumulation of technical debt
- Enforces consistent coding style across team
- Catches common programming errors early

SAST (CodeQL)

- Detects security vulnerabilities in source code
- Identifies OWASP Top 10 issues (SQL injection, XSS, etc.)
- Shift-left security - finding issues before deployment

SCA (OWASP Dependency Check)

- Scans third-party dependencies for known vulnerabilities
- Protects against supply chain attacks
- Tracks CVEs in project dependencies

Container Scanning (Trivy)

- Scans container images for OS and library vulnerabilities
- Prevents shipping containers with known security issues
- Provides comprehensive vulnerability reports

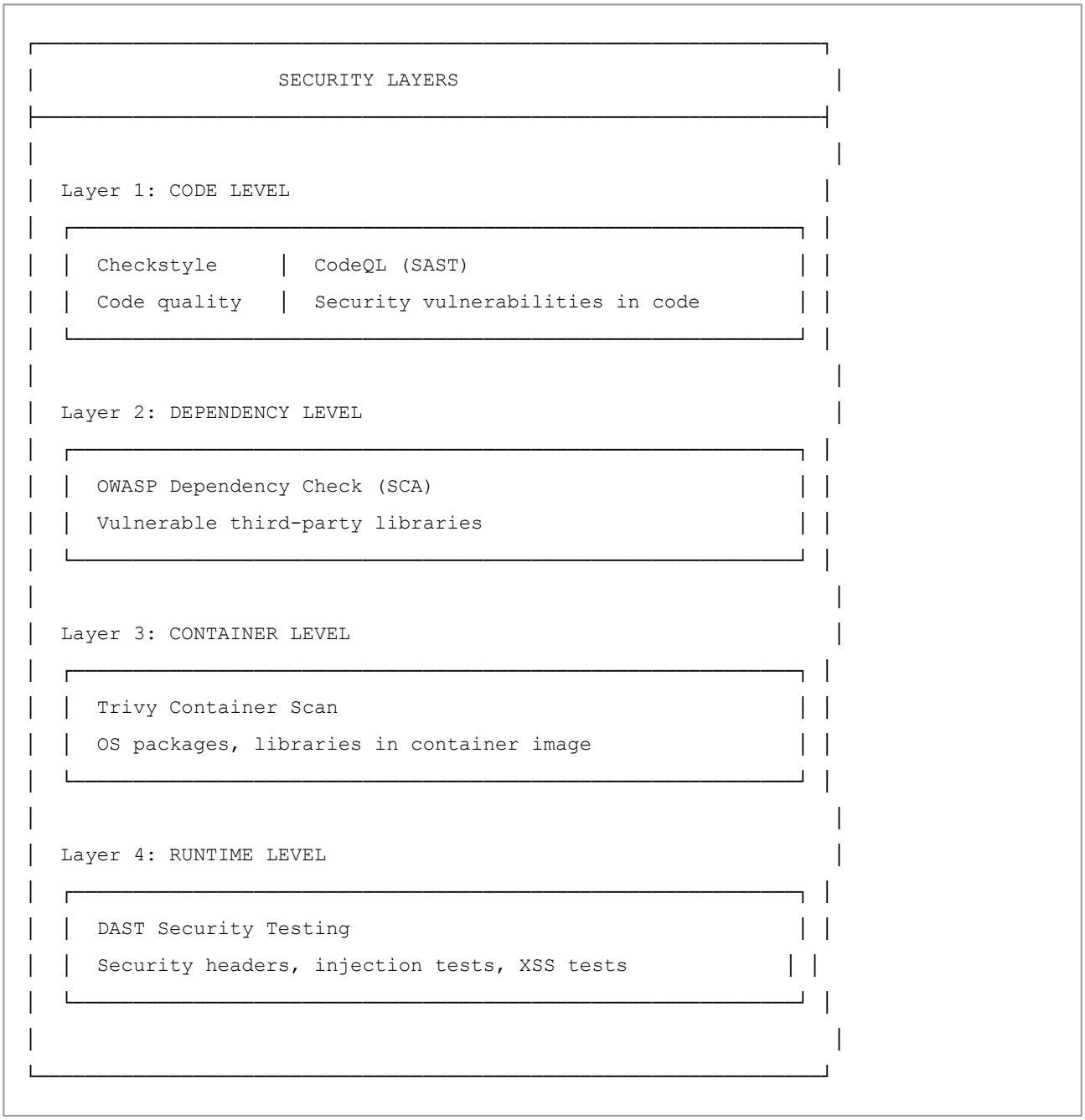
4.3 CD Pipeline Stages

#	Stage	Purpose
1	Kubernetes Deploy	Deploy application to K8s cluster
2	DAST	Dynamic Application Security Testing

---

5. Security & Quality Controls

5.1 Multi-Layer Security Approach



5.2 Quality Gates

Gate	Criteria	Action on Failure
Checkstyle	Style violations	Warning (report)
CodeQL	Security findings	Report to Security tab
Dependency Check	CVSS >= 7	Warning (report)
Unit Tests	Test failures	Block pipeline
Trivy	Critical/High vulns	Warning (report)
Smoke Test	Container not running	Block pipeline

5.3 Shift-Left Security

Traditional security testing happens late in the development cycle. Our pipeline implements **shift-left security**:

```
Traditional:      Code → Build → Test → ... → Security → Deploy
                                                (Late, expensive)

Shift-Left:       Code → Security → Build → Test → Deploy
                         (Early, cheap to fix)
```

---

6. Results & Observations

6.1 Pipeline Execution Results

The CI/CD pipeline successfully implements:

 Automated Builds: Every push triggers complete build process **Test Automation**: 8 unit tests execute automatically
 Security Scanning: Three layers of security analysis **Container Build**: Multi-stage Docker builds with optimization
 Kubernetes Ready: Deployment and service manifests prepared

6.2 Key Metrics

Metric	Value
Total CI Stages	11
Security Scan Layers	4
Unit Test Cases	8
Kubernetes Replicas	2
Auto-scaling Range	2-5 pods

6.3 Benefits Achieved

1. **Early Detection**: Security issues caught at code commit
2. **Consistent Builds**: Same process runs every time
3. **Fast Feedback**: Developers know immediately if changes break something
4. **Audit Trail**: Complete history of all pipeline runs
5. **Production Ready**: Container images verified before deployment

---

7. Limitations & Future Improvements

7.1 Current Limitations

Limitation	Description
In-memory Storage	Application uses in-memory data (no persistence)
Dry-run K8s Deploy	Kubernetes deployment runs in dry-run mode
No Integration Tests	Only unit tests implemented

Limitation	Description
Single Environment	No staging/production separation

7.2 Recommended Improvements

Short-term:

- Add database integration (PostgreSQL/MySQL)
- Implement integration tests
- Add API documentation with Swagger/OpenAPI

Medium-term:

- Implement GitOps with ArgoCD
- Add staging environment pipeline
- Implement canary deployments

Long-term:

- Service mesh integration (Istio)
- Observability stack (Prometheus, Grafana, Jaeger)
- Chaos engineering tests

---

8. Conclusion

This project demonstrates a **production-grade CI/CD pipeline** that embodies DevOps best practices:

1. **Automation First**: Every step from code to deployment is automated
2. **Security Built-In**: Multiple layers of security scanning at every stage
3. **Fail-Fast Approach**: Issues detected and reported early
4. **Clear Documentation**: Every stage has documented purpose and reasoning

The pipeline follows **DevSecOps principles** by integrating security at every stage rather than treating it as an afterthought. This shift-left approach ensures vulnerabilities are caught when they're cheapest to fix.

Appendix A: File Structure

```
project-root/
├── .github/workflows/
│   ├── ci.yml                    # CI pipeline configuration
│   └── cd.yml                    # CD pipeline configuration
├── k8s/
│   ├── deployment.yaml           # Kubernetes Deployment
│   └── service.yaml              # Kubernetes Service
├── src/
│   ├── main/java/com/taskmanager/
│   │   ├── Application.java      # Spring Boot entry point
│   │   ├── controller/
│   │   │   └── TaskController.java
│   │   ├── model/
│   │   │   └── Task.java
│   │   └── service/
│   │       └── TaskService.java
│   ├── main/resources/
│   │   └── application.properties
│   └── test/java/com/taskmanager/
│       └── TaskServiceTest.java
├── Dockerfile                    # Multi-stage Docker build
├── .dockerignore
├── pom.xml                       # Maven configuration
├── checkstyle.xml               # Code style rules
└── README.md                     # Project documentation
```

Appendix B: GitHub Secrets

Secret Name	Purpose
DOCKERHUB_USERNAME	Docker registry authentication
DOCKERHUB_TOKEN	Docker registry access token